GeoPulse

Hyper-Local Retail Mobility Analytics

PROJECT README

Retail Strategy & Geospatial Analytics

Purpose

A geospatial analytics prototype that transforms anonymized location pings into hourly footfall and store-overlap insights, helping retailers evaluate potential store locations.

Project README and four-week implementation guide

1. Project Overview

GeoPulse helps retailers make better store-location decisions using anonymized mobile GPS movement data. Rather than relying only on static census data and population density, it analyzes location pings over time to understand how people move through a city.

The pipeline processes GPS coordinates, connects them to store catchment areas, calculates hourly footfall, and visualizes movement patterns through an interactive geospatial dashboard.

2. Business Problem

Retailers may select new locations without understanding actual movement patterns. This can result in low-footfall sites, overlapping catchments, customer diversion between nearby stores, missed time-of-day patterns, and costly real-estate decisions.

GeoPulse addresses these issues through time-based spatial analysis.

3. Project Objectives

- Generate synthetic anonymized GPS pings.
- Store geographic coordinates using Snowflake.
- Create 500-meter store catchment areas.
- Perform spatial joins using PySpark and Apache Sedona.
- Use dbt to produce hourly footfall metrics.
- Identify devices appearing in multiple store catchments.
- Calculate potential audience overlap and cannibalization indicators.
- Build a time-based Kepler.gl dashboard.
- Automate daily processing with Apache Airflow.

4. Technology Stack

Technology	Purpose
Python	Data generation and analysis
Pandas / NumPy	Synthetic data preparation
Snowflake / GEOGRAPHY	Geospatial storage
PySpark / Apache Sedona	Distributed spatial processing
SQL / dbt	Analysis, transformation and tests
H3	Hexagonal spatial aggregation
Kepler.gl / React	Geospatial dashboard
Apache Airflow	Pipeline orchestration
Git / GitHub	Version control and collaboration

5. Proposed Project Architecture

- **Data generation:** Create synthetic DeviceID, latitude, longitude and timestamp records.
- **Ingestion:** Load GPS pings and store locations into Snowflake.
- **Geospatial preparation:** Create geographic points and store catchment polygons.
- **Spatial processing:** Use PySpark and Sedona to match points to catchments.
- **Transformation:** Use dbt to clean, validate and aggregate spatial matches.
- **Mobility analysis:** Calculate hourly visitors, unique devices and overlap.

- **Visualization:** Display footfall, catchments and time-based patterns in Kepler.gl.
- **Automation:** Schedule the pipeline using Airflow.

6. Suggested Repository Structure

```
GeoPulse-Hyperlocal-Retail-Analytics/
■■■ data/{raw,processed}/
■■■ src/
■   ■■■ generate_gps_data.py
■   ■■■ generate_store_locations.py
■   ■■■ validate_data.py
■■■ snowflake/{create_tables.sql,load_data.sql}
■■■ spatial_processing/
■   ■■■ create_catchment_areas.py
■   ■■■ spatial_join.py
■   ■■■ spatial_validation.py
■■■ dbt_geopulse/
■   ■■■ models/{staging,intermediate,marts}/
■   ■■■ tests/
■   ■■■ dbt_project.yml
■■■ analytics/
■   ■■■ hourly_footfall.sql
■   ■■■ visitor_overlap.sql
■   ■■■ cannibalization_analysis.sql
■■■ dashboard/{public,src}/
■■■ airflow/dags/geopulse_daily_pipeline.py
■■■ tests/
■■■ docs/{architecture.md,data_dictionary.md,methodology.md}
■■■ requirements.txt
■■■ .gitignore
■■■ README.md
```

7. Dataset Design

Dataset / Column	Description
GPS: DeviceID	Anonymized synthetic device identifier
GPS: Latitude / Longitude	GPS coordinates
GPS: Timestamp	Date and time of ping
Store: StoreID / StoreName	Store identifier and name
Store: Latitude / Longitude	Store coordinates
Store: StoreType	Existing or proposed store
Spatial: DeviceID / StoreID	Device and catchment matched
Spatial: Timestamp	Time of ping
Spatial: DistanceMeters	Distance from store
Spatial: VisitDate / VisitHour	Date and hour of observation

A GPS ping inside a catchment area indicates potential presence, not necessarily a confirmed store visit or purchase.

8. Week-wise Development Plan

Week 1 – Data Generation and Snowflake Setup

Spatial engineering:

- Understand the GPS dataset structure.
- Generate synthetic device IDs, coordinates and timestamps.
- Simulate movement across selected city areas.
- Generate existing and proposed store locations.
- Validate coordinate ranges and timestamps.

Data loading:

- Create a Snowflake account or use an approved environment.
- Create a database and schema.
- Create raw GPS and store tables with GEOGRAPHY columns.
- Load sample data and verify row counts.

Deliverables: GPS dataset, store dataset, Snowflake scripts, data dictionary and initial data-quality report.

Week 2 – Spatial Joins and dbt Modeling

- Create 500-meter catchment areas around stores.
- Configure PySpark and Apache Sedona.
- Join GPS points to store catchment polygons.
- Validate a small sample before scaling.
- Configure dbt with Snowflake and create staging/intermediate models.
- Aggregate unique devices by store and hour.
- Add tests for required fields, unique keys and valid values.

Mid-project review: Record input points, spatial matches, unmatched points, morning/evening footfall, and runtime/memory usage for the tested data size.

Deliverables: Spatial join output, dbt models and tests, hourly footfall table, and data-integrity report.

Week 3 – Cannibalization Analysis and Dashboard

- Identify anonymized devices appearing in multiple store catchments.
- Compare store overlap by date and hour.
- Calculate unique visitor counts per store.
- Estimate potential overlap between existing and proposed stores.
- Build a React application and integrate Kepler.gl.
- Display store locations and catchments; add hourly filters and a time slider.
- Visualize footfall using H3 hexagons.

Deliverables: Visitor-overlap SQL models, cannibalization analysis, React/Kepler.gl prototype and initial mobility dashboard.

Week 4 – Automation and Final Review

- Create an Airflow DAG for ingestion, spatial processing and dbt.
- Add logging, failure handling and rerun validation.
- Configure 3D H3 hexagons and a 24-hour slider.
- Improve map legends, filters and tooltips.

- Validate dashboard figures against source tables.
- Run tests, capture screenshots, update documentation and prepare the demo.

Deliverables: Integrated prototype, automated pipeline, final map, test results, technical documentation and final presentation.

9. Key Business Metrics

Metric	Meaning
Total GPS Pings	Total location observations
Unique Devices	Distinct anonymized devices observed
Hourly Footfall	Unique devices detected in a store catchment during an hour
Catchment Coverage	Devices observed within a store catchment
Visitor Overlap	Devices detected in multiple store catchments
Potential Cannibalization	Estimated share of a target store's relevant audience overlapping an existing store
Morning / Evening Footfall	Unique devices observed in defined time periods

Methodology note: Visitor overlap is not proof of cannibalization. A device may pass through multiple areas without visiting either store. A defensible estimate needs a defined time window, audience denominator, visit-detection rules, and ideally sales or conversion data.

10. Data Privacy and Responsible Use

- Use synthetic or appropriately anonymized location data.
- Do not expose personally identifiable information or attempt to identify people from movement.
- Apply access controls and data-retention limits.
- Clearly label simulated results and assumptions.

11. Expected Final Outcome

GeoPulse will provide a reproducible geospatial analytics pipeline that transforms location pings into hourly footfall and store-overlap insights. Decision-makers can compare catchments, inspect patterns throughout the day, and evaluate whether a proposed location may overlap with an existing store's audience.

This is an analytical decision-support prototype. Validate outputs against real visits and sales before using them for real-estate investment decisions.

12. How to Run the Project

The exact commands depend on installed tools and configured credentials. Intended sequence:

- Install dependencies from requirements.txt.
- Generate synthetic GPS and store datasets.
- Configure Snowflake credentials securely; create database, schema and tables.
- Load raw datasets.
- Configure and run the PySpark/Sedona spatial join.
- Configure dbt profiles and run models and tests.
- Start the React/Kepler.gl dashboard.
- Configure and trigger the Airflow DAG.

Never commit credentials, passwords, private keys, or real personal location data to GitHub.

13. Project Status

This README describes the proposed design and implementation plan. Update the status as work is completed. Do not mark Snowflake, Sedona, dbt, React, Kepler.gl, or Airflow as implemented until each component has been run and validated.

GeoPulse – Hyper-Local Retail Mobility Analytics
Domain: Retail Strategy and Geospatial Analytics
Goal: Use time-based mobility patterns to support better retail location decisions.